



Verilənlər Bazası Arxitekturası

Sharding · Partitioning · Replikasiya

Nədir · Növləri · Üstünlükləri · Çatışmazlıqları · Real Nümunələr

2025 · Tam Azərbaycan Dilli Texniki Bələdçi

□ Mündəricat

1. Giriş — Niyə Bu Mövzular Vacibdir?

2. Partitioning

2.1 Horizontal

2.2 Vertical

2.3 Range

2.4 Hash

2.5 List

2.6 Composite

3. Sharding — Dərin İzah

4. Sharding Strategiyaları

4.1 Key-Based

4.2 Range-Based

4.3 Directory-Based

4.4 Geo-Based

5. Sharding: Üstünlüklər (+)

6. Sharding: Çatışmazlıqlar (–)

7. Replikasiya — Giriş və Əsas Anlayışlar

8. Replikasiya Növləri

8.1 Master-Slave

8.2 Master-Master

8.3 Synchronous

8.4 Asynchronous

8.5 Semi-Synchronous

8.6 Chain / Cascaded

8.7 Multi-Region

9. Replikasiya: Üstünlüklər (+)

10. Replikasiya: Çatışmazlıqlar (–)

11. Partitioning vs Sharding vs Replikasiya — Müqayisə

12. Real Həyat Nümunələri

13. Hansı DB-lər Nəyi Dəstəkləyir?

14. Nə Vaxt Nəyi Seçməli?

15. Nəticə

1. Giriş — Niyə Bu Mövzular Vacibdir?

Müasir veb tətbiqləri milyonlarla istifadəçiyə xidmət edir. Instagram-da gündəlik 100 milyondan çox şəkil yüklənir, Twitter-də saniyədə minlərlə tvit atılır. Bu cür yükü tək bir verilənlər bazası serveri daşıya bilmir.

Bu problemi həll etmək üçün üç əsas arxitektura konsepti var: Partitioning, Sharding və Replikasiya. Hər biri fərqli problemi həll edir və çox vaxt birlikdə istifadə olunur.

□ 3 Konseptin Fərqi

Partitioning: Məlumatı eyni serverdə məntiqi hissələrə bölmək | Sharding: Məlumatı müxtəlif fiziki serverlərə (node) paylamaq | Replikasiya: Eyni məlumatın bir neçə serverdə kopyasını saxlamaq

2. Partitioning Nədir?

Partitioning, böyük bir cədvəl məntiqi/fiziki olaraq daha kiçik hissələrə — partition-lara — ayırmaqdır. Bu hissələr eyni verilənlər bazası serverində saxlanılır.

2.1 Horizontal Partitioning (Cərgə Bölməsi)

Cədvəlin sətirləri müəyyən kriterlərə görə fərqli bölmələrə paylanır. Hər bölmə eyni sütun strukturuna malikdir.

Nümunə: Orders → orders_2023 | orders_2024 | orders_2025

2.2 Vertical Partitioning (Sütun Bölməsi)

Cədvəlin ağır sütunları (BLOB, TEXT) ayrı cədvəldə saxlanılır ki, əsas cədvəl sürətlə oxunsun.

users(id, name, email) + users_profile(user_id, bio, avatar_blob)

2.3 Range Partitioning

Sütunun dəyər aralığına görə bölmə aparılır. Tarix, ID üçün idealdır.

PARTITION BY RANGE (YEAR(created_at)): 2022 | 2023 | 2024 | 2025

2.4 Hash Partitioning

Hash funksiyası tətbiq edilir, nəticə bölmə sayına bölünür. Bərabər paylama üçün idealdır.

partition_no = hash(user_id) % 4 → 0, 1, 2 və ya 3

2.5 List Partitioning

Dəyərlərin konkret siyahısına görə bölmə: ölkə, kateqoriya, status üçün uyğundur.

PARTITION BY LIST(country): AZ → p_az | TR → p_tr | US → p_us

2.6 Composite Partitioning

İki strategiyanın kombinasiyası — məs. əvvəl Range (il), sonra Hash (user_id). PostgreSQL, MySQL dəstəkləyir.

3. Sharding — Dərin İzah

Sharding, verilənlər bazasını shard adlanan hissələrə bölərək hər birini ayrı fiziki serverə yerləşdirməkdir. Hər shard öz verilənlər bazası instance-ına malikdir.

İstifadəçi sorğu göndərdikdə Shard Router, Shard Key-ə əsasən hansı node-a yönləndirəcəyini bilir. Bu proses istifadəçiyə şəffafdır.

Tətbiq	→ Router →	Shard 0 (Server A)	Shard 1 (Server B)	Shard 2 (Server C)
user_id=500	hash % 3 = 2			✓ Burada
user_id=101	hash % 3 = 0	✓ Burada		

⚠ Shard Key Seçimi

Shard key seçimi ən kritik qərardır. Yanlış seçim 'hotspot' — bir shard-ın həddindən artıq yüklənməsi — probleminə səbəb olur. Yüksək kardinaliteli (çox unikal dəyər), bərabər paylanan sütunları seçin.

4. Sharding Strategiyaları

4.1 Key-Based (Hash) Sharding

$shard_id = hash(user_id) \% total_shards$

- Məlumatlar bərabər paylanır. Hash deterministikdir — eyni key həmişə eyni shard-a gedir.
- Yeni shard əlavə etdikdə rehashing lazımdır. Consistent Hashing bu problemi həll edir.

4.2 Range-Based Sharding

$user_id\ 1-10000 \rightarrow Shard\ A \mid 10001-20000 \rightarrow Shard\ B \mid 20001+ \rightarrow Shard\ C$

- Range sorğuları effektivdir. Arxiv məlumatları üçün köhnə shard-ı asanlıqla drop etmək olur.
- Son shard daima dolur — hotspot riski. Monoton artan key-lər ilə problem yaranır.

4.3 Directory-Based Sharding

Lookup: $user_id \rightarrow shard_id$ (Redis və ya ayrı DB-də saxlanılır)

- Ən çevik üsul — istənilən məntiq tətbiq etmək olur. Migration asandır.
- Lookup xidməti bottleneck ola bilər. SPOF (Single Point of Failure) riskidir.

4.4 Geo-Based Sharding

$Europe \rightarrow EU\ Shard \mid US \rightarrow US\ Shard \mid Asia \rightarrow APAC\ Shard$

- Latency minimaldır. GDPR kimi data residency tələbləri ödənilir.
- VPN istifadəçiləri yanlış shard-a yönlənə bilər. Shard-lar arası disbalans.

5. Sharding: Üstünlüklər (+)

➤ **Yüksək Performans**

Hər shard yalnız öz hissəsini emal edir. Paralel emal mümkündür, I/O yükü bölünür.

□ **Horizontal Miqyaslama**

Yük artdıqca yeni serverlər əlavə etmək kifayətdir. Hardware limitinə çatmaq riski aradan qalxır.

□ **Yüksək Əlçatanlıq**

Bir shard-ın xarab olması digər shard-ları etkiləmir. Replication ilə birlikdə sistem dayanmadan işləyir.

□ **Data İzolyasiyası**

Multi-tenant SaaS tətbiqlərində hər müştərinin məlumatı fiziki olaraq ayrı saxlanılır.

□ **Coğrafi Yaxınlıq**

İstifadəçilərə ən yaxın data mərkəzindəki shard-a yönləndirməklə gecikməni minimuma endirmək olur.

□ **Sürətli Silmə**

Range partition ilə köhnəlmiş shard-ı birbaşa DROP etmək olur — DELETE-dən çox sürətlidir.

6. Sharding: Çatışmazlıqlar (–)

⚠ **Kompleks Arxitektura**

Shard router, shard key seçimi, resharding planlaması — bütün bunlar əlavə mühəndislik işi tələb edir.

❏ **Cross-Shard JOIN**

Müxtəlif shard-lardakı cədvəlləri JOIN etmək ya qeyri-mümkündür, ya da tətbiq səviyyəsində merge lazımdır.

❏ **Hotspot Riski**

Yanlış shard key seçimi ilə bütün trafik bir shard-a axır. Bərabər paylama kritikdir.

❏ **Distributed Transactions**

Çox shard-ı əhatə edən ACID əməliyyatları üçün 2-Phase Commit (2PC) lazımdır — performans aşağı düşür.

❏ **Resharding Çətinliyi**

Shard key sonradan dəyişdirilə bilmir. Yanlış seçim bütün məlumatların yenidən paylanmasını tələb edir.

❏ **Unikal ID Problemi**

Auto-increment ID-lər çox shard-da konflikt yaradır. UUID, Snowflake ID, ULID istifadə edilməlidir.

7. Replikasiya — Giriş və Əsas Anlayışlar

Replikasiya (Replication), eyni verilənlər bazasının məlumatlarının bir və ya bir neçə əlavə serverə (replica) avtomatik olaraq kopyalanması prosesidir. Şarding məlumatı böldüyü halda, replikasiya məlumatı çoxaltır.

Məqsədlər bunlardır: (1) Yüksək Əlçatanlıq (High Availability) — əsas server çökdükdə replica dərhal devreye girsin; (2) Read Scalability — oxuma sorğuları replica-lara paylansın; (3) Disaster Recovery — fiziki fəlakət zamanı məlumat itkisi olmasın; (4) Coğrafi Yaxınlıq — müxtəlif regionlarda replica saxlamaq.

Primary (Master)	Write Ahead Log (WAL)	Replica (Slave)
INSERT, UPDATE, DELETE qəbul edir	→ dəyişiklikləri ötürür →	Yalnız SELECT (oxuma) edir

Əsas terminlər: Primary / Master — yazma əməliyyatlarını qəbul edən əsas server. Replica / Slave / Standby — primary-dən məlumat alan köməkçi server(lər). WAL (Write-Ahead Log) — PostgreSQL-də dəyişikliklərin jurnal faylı; replikasiya bununla işləyir. Binlog — MySQL-də eyni məqsədli jurnal. Replication Lag — primary-dəki dəyişikliyin replica-ya çatma gecikməsi.

8. Replikasiya Növləri

8.1 Master-Slave (Primary-Replica) Replikasiya

Ən geniş yayılmış model. Bir primary bütün yazmaları qəbul edir, bir və ya bir neçə replica oxuma üçün istifadə olunur. Primary xarab olduqda manuel və ya avtomatik failover ilə replica primary-ə çevrilir.

Xüsusiyyət	Dəyər
Yalnız Primary	
Primary + Replica-lar	
Manuel və ya avtomatik (Patroni, MHA)	
Async modda: bəli (Replication Lag qədər)	
PostgreSQL Streaming, MySQL Binlog	

PostgreSQL: primary_conninfo = 'host=primary_ip port=5432' recovery.conf / standby.signal

8.2 Master-Master (Multi-Primary) Replikasiya

Birdən çox node həm yazma, həm oxuma qəbul edir. Hər node eyni anda primary rolunu oynayır. Yüksək əlçatanlıq tələb edən, çox regionlu sistemlər üçün istifadə edilir.

Ən böyük problem: Write Conflict — iki node eyni anda eyni sıraya fərqli dəyər yazdıqda hansının düzgün olduğunu müəyyən etmək lazımdır. Conflict Resolution strategiyaları: Last-Write-Wins (LWW), Timestamp-based, Application-level merge.

⚠ Multi-Master Xəbərdarlığı

Multi-master replikasiya yüksək mühəndislik resurs tələb edir. Conflict resolution yanlış konfigurasiya edilsə data itkisi baş verə bilər. Alternativ olaraq CockroachDB, Spanner kimi Distributed SQL-ə baxın.

8.3 Synchronous (Sinxron) Replikasiya

Primary dəyişikliyi yazdıqdan sonra ən azı bir replica-nın təsdiqini (ACK) gözləyir, ancaq o zaman client-ə 'uğurlu' cavabı qaytarır. Bu zəmanət verir ki, primary çöxsə belə, replica-da ən son məlumat var.

PostgreSQL: `synchronous_commit = on synchronous_standby_names = 'replica1'`

- ☐ Zero data loss — məlumat itkisi sıfırdır
- ☐ ACID zəmanəti tam qorunur
- ☐ Yazma latency-si artır (replica-nın ACK-ini gözləmək lazımdır)
- ☐ Replica yavaşlarsa/çökərsə primary də yavaşlayır

8.4 Asynchronous (Asinxron) Replikasiya

Primary dəyişikliyi yazır, client-ə dərhal 'uğurlu' cavabı qaytarır, replica-ya kopyalamaq arxa planda baş verir. Replica bir qədər geri qala bilər — bu fərqə Replication Lag deyilir.

PostgreSQL: `synchronous_commit = off (default)` MySQL: `default replication mode`

- ☐ Yazma performansını maksimaldır — heç bir gözləmə yoxdur
- ☐ Replica gecikmələri primary-ə təsir etmir
- ☐ Primary çöxsə Replication Lag qədər məlumat itkisi ola bilər
- ☐ Replica-dan oxunan məlumat bir qədər köhnə ola bilər (Stale Read)

8.5 Semi-Synchronous Replikasiya

Sinxron və asinxron modellərin kompromisi. Primary ən azı bir replica-nın ACK-ini gözləyir, amma bütün replica-ların deyil. ACK zamanı aşılırsa asinxrona keçir. MySQL 5.5+ dəstəkləyir.

MySQL: `rpl_semi_sync_master_enabled = 1 rpl_semi_sync_master_timeout = 1000 (ms)`

Bu model məlumat itkisi riskini minimuma endirir, eyni zamanda performans cəzasını tam sinxronlardan az edir.

8.6 Chain / Cascaded Replikasiya

Replica-lar zəncir şəklində yerləşdirilir: Primary → Replica-1 → Replica-2 → Replica-3. Replica-1 həm primary-dən qəbul edir, həm özü Replica-2-yə ötürür. Bu üsul primary-dəki replikasiya yükünü azaldır.

PostgreSQL: `recovery_min_apply_delay` — Replica-2-nin Replica-1-ə gecikmə tənzimlənməsi

- Primary üzərindəki replikasiya yükü azalır
- Çox sayda replica saxlamaq üçün idealdır
- Zəncirdəki ara node çökərsə, sonrakı replica-lar da kəsilir
- Replication Lag zəncir uzandıqca artır

8.7 Multi-Region (Coğrafi) Replikasiya

Replica-lar müxtəlif coğrafi regionlarda (data mərkəzlərində) saxlanılır. Disaster Recovery üçün kritikdir. İstifadəçilər ən yaxın regiondan oxuya bilir.

Region	Rol	Gecikməsi	İstifadəsi
Bakı (AZ)	Primary	0 ms	Yazma + Oxuma
Frankfurt (DE)	Sync Replica	~60 ms	Failover + EU Oxuma
Virginia (US)	Async Replica	~120 ms	US Oxuma + DR
Singapore (SG)	Async Replica	~150 ms	APAC Oxuma + DR

9. Replikasiya: Üstünlüklər (+)

☐ **Yüksək Əlçatanlıq (High Availability)**

Primary xarab olduqda replica dərhal devreye girə bilər. Məhsul xidmət kəsilişi minimuma enir. Patroni, Orchestrator kimi alətlər bu prosesi avtomatlaşdırır.

☐ **Read Scalability (Oxuma Miqyaslanması)**

Oxuma sorğuları replica-lara paylana bilər. Analitik sorğular, hesabatlar, dashboard-lar primary-ə yük vurmada replica-lardan işləyə bilər.

☐ **Disaster Recovery (Fəlakətin Bərpası)**

Fiziki fəlakət, yanğın, data mərkəzi nasazlığı zamanı başqa regiondakı replica məlumatları qoruyur. RPO (Recovery Point Objective) minimaldır.

☐ **Coğrafi Yaxınlıq**

İstifadəçilərə ən yaxın regiondakı replica-dan oxumaq latency-ni əhəmiyyətli dərəcədə azaldır. Qlobal SaaS platformaları üçün vacibdir.

☐ **Texniki Xidmət Asanlıqı**

Primary-də texniki işlər aparılarkən traffic replica-ya yönləndirilə bilər. Zero-downtime maintenance mümkün olur.

☐ **Analitik Sorğu İzolyasiyası**

Ağır OLAP sorğularını replica-ya yönləndirərək production primary-inin performansını qorumaq olur.

☐ **Point-in-Time Recovery**

WAL/Binlog arxivi saxlandıqda istənilən keçmiş anına qayıtmaq mümkündür. Yanlış DELETE/UPDATE-dən sonra məlumatları bərpa etmək olur.

10. Replikasiya: Çatışmazlıqlar (–)

❑ Replication Lag (Gecikməsi)

Asinxron modda replica-da məlumat bir qədər köhnə ola bilər. İstifadəçi yazdığı məlumatı dərhal oxumaq istəsə 'stale read' problemi yaşana bilər. Read-your-writes consistency pozulur.

❑ Əlavə Xərc

Hər replica ayrıca server, lisenziya, əməliyyat xərci deməkdir. Bir primary üçün 3 replica saxlamaq infra xərclərini 4x artırır.

❑ Failover Mürəkkəbliyi

Primary çöküşü zamanı hansı replica-nın yeni primary olacağı, digər replica-ların ona yönləndirilməsi, client connection string-lərinin yenilənməsi koordinasiya tələb edir.

📌 Yazma Bottleneck

Yazma əməliyyatları hər zaman primary-ə gedir — bu da scale out-u məhdudlaşdırır. Multi-master həll kimi görünsə də conflict problemi gətirir.

❑ Split-Brain Problemi

Network partition zamanı hər iki node özünü primary hesab edə bilər. Bu halda iki ayrı yazma axını yaranır — data inconsistency baş verir. Quorum-based sistemlər (Raft, Paxos) bu problemi həll edir.

❑ Disk Tutumu

Hər replica tam məlumatın kopyasını saxlayır. 10TB verilənlər bazası üçün 3 replica 30TB disk istifadə edir. Sharding ilə birlikdə istifadə bu problemi azaldır.

❑ Schema Dəyişiklikləri

DDL (ALTER TABLE) əməliyyatları replica-larda bloklanma yarada bilər. Online Schema Change alətləri (pt-online-schema-change, gh-ost) tələb olunur.

11. Partitioning vs Sharding vs Replikasiya — Müqayisə

Xüsusiyyət	Partitioning	Sharding	Replikasiya
Məqsəd	Performans	Scale Out	HA + Oxuma Yüku
Server sayı	1	Çox	Çox (kopya)
Məlumat	Bölünür	Bölünür	Çoxaldılır
Yazma Yeri	Tək cədvəl	Shard-lara görə	Yalnız Primary
Oxuma Yeri	Tək server	Shard-lara görə	Primary + Replica
JOIN	Tam dəstəklənir	Çox çətindir	Tam dəstəklənir
Failover	Yoxdur	Qismən	Tam dəstəklənir
Data itkisi riski	Yox	Yox	Async modda: az
Mürəkkəblilik	Aşağı-Orta	Yüksək	Orta
İstifadə	OLAP, Arxiv	Böyük OLTP	HA, DR, Read Scale

□ Birlikdə istifadə

Bu üç texnologiya bir-birini tamamlayır. Tipik production arxitekturasında: Partitioning (cədvəl performansı üçün) + Sharding (horizontal scale üçün) + Replikasiya (HA və read scale üçün) birlikdə tətbiq edilir. Məsələn: hər shard-ın öz primary + replica-ları var.

12. Real Həyat Nümunələri

□ Twitter / X

Sharding: Snowflake ID ilə hash sharding. Replikasiya: Hər shard üçün async replica-lar. Timeline servisi eventual consistency ilə işləyir — bir neçə saniyə gecikmə qəbul edilir.

□ Instagram

Sharding: PostgreSQL + Citus — user_id-yə görə 512 virtual shard, 8 fiziki server. Replikasiya: Hər shard üçün hot standby replica. Fotoları S3-də saxlayır, yalnız metadata DB-dədir.

□ WhatsApp

Sharding: chat_id-yə görə — bir söhbətin bütün mesajları eyni shard-dadır. Replikasiya: Erlang/Mnesia ilə built-in distributed replikasiya. 2 milyard istifadəçi üçün end-to-end encrypted.

□ Amazon RDS / Aurora

Aurora: Paylaşılan storage layer üzərindən 6-yollu replikasiya (3 AZ, hər birində 2 kopya). Yazma yalnız Primary, oxuma üçün 15-ə qədər Read Replica. Multi-master Aurora xüsusi konfigurasiya tələb edir.

□ MongoDB Atlas

Sharding: Chunk-based auto-sharding. Balancer həcm artdıqca avtomatik yenidən paylayır. Replikasiya: Replica Set — minimum 3 node (1 primary + 2 secondary). Raft-əsaslı leader election ilə failover ~10 saniyədə baş verir.

□ PostgreSQL + Patroni

Ən geniş yayılmış PostgreSQL HA həlli. Patroni + etcd/ZooKeeper + HAProxy birliyi: Primary çöküşündə Patroni avtomatik yeni primary seçir, HAProxy trafiği yönləndirir. IdeaBank kimi layihələr üçün ideal açıq mənbəli həldir.

13. Hansı DB-lər Nəyi Dəstəkləyir?

Verilənlər Bazası	Partitioning	Sharding	Replikasiya	Qeyd
PostgreSQL	☐ Tam	Citus/Vitess	☐ Streaming WAL	Patroni ilə HA
MySQL	☐ Tam	Vitess	☐ Binlog	Semi-sync dəstəyi
MongoDB	☐ Daxili	☐ Auto	☐ Replica Set	Raft-based election
Cassandra	☐ Daxili	☐ Consistent Hash	☐ Multi-DC	Tunable consistency
CockroachDB	☐ Auto	☐ Auto	☐ Raft	Distributed SQL
Redis Cluster	N/A	☐ Hash Slots	☐ Async	16384 slot
Amazon Aurora	☐ Tam	Aurora Shard	☐ 6-yollu	3 AZ, 15 Read Replica
MS SQL Server	☐ Tam	Elastic DB	☐ AlwaysOn AG	Azure Managed

14. Nə Vaxt Nəyi Seçməli?

Partitioning istifadə et, əgər:

- Cədvəlin ölçüsü 100GB+ olduğunda
- Zaman seriyası məlumatları var (logs, metrics)
- Köhnə məlumatları sürətlə silmək lazımdır
- Tək server kifayət edir, amma sorğular yavaşlayır

Sharding istifadə et, əgər:

- DB tək serverin disk/RAM limitini keçib
- Saniyədə on minlərlə yazma/oxuma var
- Multi-region deployment lazımdır
- Tenant izolyasiyası tələb olunur

Replikasiya istifadə et, əgər:

- Yüksək əlçatanlıq (99.9%+) tələb olunur
- Oxuma yükü yazmadan çox qat artıqdır
- Disaster Recovery planı vacibdir
- Analitik sorğuları production-dan izolə etmək lazımdır
- Zero-downtime maintenance vacibdir

📌 Qızıl Qayda

Əksər production sistemlərində bu üçü birlikdə tətbiq edilir: Partitioning (cədvəl performansı) + Sharding (scale out) + Replikasiya (HA). Lakin kiçik sistemlər üçün yalnız Replikasiya (Patroni + PostgreSQL) çox vaxt kifayətdir. Mürəkkəbliyi yalnız real ehtiyac varsa artırın.

15. Nəticə

Partitioning, Sharding və Replikasiya — müasir verilənlər bazası arxitekturasının üç sütunudur. Hər biri fərqli problemi həll edir və bir-birini tamamlayır.

Partitioning — tək server çərçivəsində sorğu performansını artırır, köhnə məlumatların idarəsini asanlaşdırır.

Sharding — əsl horizontal miqyaslanma tələb edən sistemlər üçün məlumatı fiziki olaraq birdən çox serverə paylayır. Shard key seçimi ən kritik qərardır.

Replikasiya — məlumatın eyni kopyasını birdən çox serverdə saxlayır. Yüksək əlçatanlıq, oxuma miqyaslanması və fəlakətin bərpası üçün vacibdir.

▣ Xülasə Cədvəli

- Partitioning = Tək serverdə bölmə | sorğu performansı + arxiv idarəsi •
- Sharding = Çoxlu serverə paylama | horizontal scale + tenant izolyasiya •
- Replikasiya = Eyni məlumatın kopyalanması | HA + read scale + DR •
- Master-Slave: sadə, geniş yayılmış | Async: sürətli, az zəmanət • Sync Replica: data itkisi sıfır | latency artır • Patroni + PostgreSQL: açıq mənbəli HA həlli üçün sənaye standartı •
- Tipik production: Partitioning + Sharding + Replikasiya birlikdə

Bu sənəd IdeaBank Backend Seriyası çərçivəsində hazırlanmışdır.